

KlinikKu: Mobile-Based Clinic Information System Using Flutter and Laravel REST API with MVC Architecture

1st Faiz Azzikri
Teknik Informatika
Universitas Muhammadiyah
Bandung
Bandung, Indonesia
Faizzazzikri@gmail.com

2nd Mochamad firdausa arfiandi
nur Pratama
Teknik Informatika
Universitas Muhammadiyah
Bandung
Tasikmalaya, Indonesia
nevingaming001@gmail.com

3rd Muhammad Fadil Rizki
Teknik Informatika
Universitas Muhammadiyah
Bandung
Lampung, Indonesia
fdlrzky8@gmail.com

4th Muhammad Rizqy Subagja
Teknik Informatika
Universitas Muhammadiyah
Bandung
Sumedang, Indonesia
rizqysubagja99@gmail.com

Abstract—Pelayanan administrasi klinik yang masih dilakukan secara manual sering menimbulkan berbagai permasalahan, seperti keterlambatan proses pendaftaran pasien, kesalahan pencatatan data, pengelolaan antrean yang kurang teratur, serta kesulitan dalam penyimpanan riwayat pemeriksaan. Penelitian ini bertujuan mengembangkan KlinikKu, yaitu sistem informasi klinik berbasis mobile yang menerapkan arsitektur client-server menggunakan Flutter sebagai aplikasi client dan Laravel sebagai backend yang berkomunikasi melalui REST API. Sistem dibangun dengan pola Model-View-Controller (MVC), menggunakan Laravel Sanctum untuk autentikasi, Eloquent ORM sebagai pengelola akses basis data, serta MySQL sebagai media penyimpanan data. Fitur utama yang dikembangkan meliputi pengelolaan data pasien, dokter, jadwal pemeriksaan, antrean, dan rekam medis yang terintegrasi dalam satu sistem. Penerapan arsitektur client-server memungkinkan pemisahan antara aplikasi mobile dan backend sehingga pengembangan maupun pemeliharaan sistem menjadi lebih terstruktur. Melalui integrasi tersebut, sistem diharapkan mampu meningkatkan efisiensi pengelolaan administrasi klinik, mempermudah akses terhadap data pasien dan riwayat pemeriksaan, serta mendukung pelayanan klinik yang lebih cepat, akurat, dan terorganisir.

Keywords—Client-Server, Flutter, Laravel, Model-View-Controller (MVC), REST API, Sistem Informasi Klinik.)

I. INTRODUCTION

A. Background

Perkembangan teknologi informasi telah mendorong transformasi digital di berbagai sektor, termasuk pada bidang pelayanan kesehatan. Pemanfaatan sistem informasi dalam

layanan klinik menjadi salah satu solusi untuk meningkatkan efektivitas pengelolaan data, mempercepat proses administrasi, serta meningkatkan kualitas pelayanan kepada pasien. Dibandingkan dengan proses manual, sistem informasi mampu mengurangi risiko kesalahan pencatatan, mempermudah penyimpanan data, dan menyediakan akses informasi yang lebih cepat bagi petugas maupun tenaga medis.

Pada praktiknya, masih banyak klinik yang menghadapi berbagai kendala dalam pengelolaan administrasi, seperti proses pendaftaran pasien yang memerlukan waktu cukup lama, pencatatan data yang rentan terhadap kesalahan, pengelolaan antrean yang kurang teratur, serta kesulitan dalam menyimpan dan menelusuri riwayat pemeriksaan pasien. Kondisi tersebut dapat memengaruhi efisiensi operasional klinik serta kualitas pelayanan yang diberikan kepada pasien. Oleh karena itu, diperlukan suatu sistem informasi yang mampu mengintegrasikan seluruh proses administrasi dan pelayanan klinik ke dalam satu platform yang terpusat.

Seiring berkembangnya teknologi perangkat bergerak, penerapan arsitektur client-server menjadi salah satu pendekatan yang banyak digunakan dalam pengembangan aplikasi modern. Pada arsitektur ini, aplikasi client bertugas sebagai antarmuka pengguna, sedangkan server bertanggung jawab menangani logika bisnis, pengelolaan data, dan komunikasi dengan basis data melalui layanan REST API. Pendekatan ini memungkinkan pengembangan frontend dan backend dilakukan secara terpisah sehingga meningkatkan fleksibilitas, skalabilitas, serta kemudahan pemeliharaan sistem.

Berdasarkan permasalahan tersebut, penelitian ini mengembangkan KlinikKu, yaitu sistem informasi klinik berbasis mobile yang dibangun menggunakan Flutter sebagai aplikasi client dan Laravel sebagai backend dengan komunikasi melalui REST API. Sistem menerapkan arsitektur Model-View-Controller (MVC) pada sisi backend serta menggunakan MySQL sebagai basis data. Fitur utama yang dikembangkan meliputi pengelolaan data pasien, dokter, jadwal pemeriksaan, antrean, dan rekam medis yang terintegrasi dalam satu sistem. Dengan penerapan arsitektur tersebut, diharapkan proses administrasi dan pelayanan klinik menjadi lebih efektif, terstruktur, serta memudahkan petugas dalam mengakses dan mengelola informasi secara cepat dan akurat.

B. Problem Statement

Berdasarkan latar belakang yang telah diuraikan, proses administrasi klinik yang masih dilakukan secara manual menimbulkan berbagai permasalahan, antara lain keterlambatan dalam pengelolaan data pasien, tingginya potensi kesalahan pencatatan, kesulitan dalam mengatur

jadwal pemeriksaan dan antrian pasien, serta kurang efektifnya penyimpanan riwayat rekam medis. Selain itu, keterbatasan integrasi antar data menyebabkan petugas harus melakukan pencatatan berulang sehingga mengurangi efisiensi pelayanan.

Pemanfaatan aplikasi mobile yang terhubung dengan backend melalui arsitektur client-server dan REST API menjadi salah satu solusi untuk mengatasi permasalahan tersebut. Namun, pengembangan sistem yang mampu mengintegrasikan modul pasien, dokter, jadwal pemeriksaan, antrian, dan rekam medis dalam satu platform yang terpusat tetap memerlukan perancangan arsitektur yang tepat agar proses pertukaran data berlangsung secara cepat, konsisten, dan mudah dipelihara.

Berdasarkan permasalahan tersebut, penelitian ini berfokus pada pengembangan sistem informasi klinik berbasis mobile menggunakan Flutter sebagai aplikasi client dan Laravel sebagai backend dengan komunikasi REST API serta penerapan arsitektur Model-View-Controller (MVC) untuk menghasilkan sistem yang terintegrasi, mudah dikembangkan, dan mampu mendukung pelayanan administrasi klinik secara lebih efektif.

C. Research Objectives

Penelitian ini bertujuan untuk mengembangkan aplikasi KlinikKu, yaitu sistem informasi klinik berbasis mobile yang menerapkan arsitektur client-server menggunakan Flutter sebagai frontend dan Laravel sebagai backend yang berkomunikasi melalui REST API. Selain itu, penelitian ini bertujuan untuk mengimplementasikan pola arsitektur Model-View-Controller (MVC) pada backend guna menghasilkan struktur aplikasi yang lebih terorganisir dan mudah dipelihara. Sistem yang dikembangkan diharapkan mampu mengintegrasikan pengelolaan data pasien, dokter, jadwal pemeriksaan, antrian, dan rekam medis sehingga proses administrasi serta pelayanan klinik menjadi lebih efektif, efisien, dan terstruktur.

D. Research Contributions

Kontribusi utama penelitian ini adalah pengembangan aplikasi KlinikKu, yaitu sistem informasi klinik berbasis mobile yang mengintegrasikan aplikasi Flutter dengan backend Laravel melalui REST API dalam arsitektur client-server. Penelitian ini juga menerapkan pola Model-View-Controller (MVC) pada backend untuk meningkatkan keteraturan pengembangan perangkat lunak serta memanfaatkan Laravel Sanctum sebagai mekanisme autentikasi API dan Eloquent ORM dalam pengelolaan basis data MySQL. Selain menghasilkan sistem yang mendukung pengelolaan data pasien, dokter, jadwal pemeriksaan, antrian, dan rekam medis secara terintegrasi, penelitian ini diharapkan dapat menjadi salah satu referensi penerapan teknologi Flutter dan Laravel dalam pengembangan sistem informasi pelayanan kesehatan berbasis mobile.

II. LITERATURE REVIEW

A. Client-Server Architecture

Arsitektur client-server merupakan salah satu model yang paling banyak digunakan dalam pengembangan sistem informasi modern. Pada arsitektur ini, aplikasi dibagi menjadi dua komponen utama, yaitu client sebagai antarmuka yang digunakan pengguna dan server sebagai penyedia layanan yang bertanggung jawab terhadap pemrosesan logika bisnis, pengelolaan data, serta komunikasi dengan basis data.

Pemisahan tanggung jawab tersebut memungkinkan pengembangan sistem menjadi lebih terstruktur, mudah dipelihara, dan memiliki skalabilitas yang lebih baik.

Dalam implementasinya, arsitektur client-server umumnya menerapkan konsep three-tier architecture yang terdiri atas presentation layer, application layer, dan data layer. Presentation layer bertugas menyediakan antarmuka pengguna, application layer menjalankan proses bisnis dan layanan aplikasi, sedangkan data layer bertanggung jawab terhadap penyimpanan serta pengelolaan data. Pemisahan ketiga lapisan tersebut memungkinkan setiap komponen dikembangkan maupun diperbarui secara independen tanpa memengaruhi keseluruhan sistem.

Pada penelitian ini, konsep three-tier architecture diterapkan dengan menggunakan Flutter sebagai presentation layer, Laravel REST API sebagai application layer, dan MySQL sebagai data layer. Melalui pembagian tersebut, komunikasi antara aplikasi mobile dan server dapat dilakukan secara efisien sehingga mendukung pengelolaan data pasien, dokter, jadwal pemeriksaan, antrian, dan rekam medis dalam satu sistem yang terintegrasi.

B. Rest Api

Representational State Transfer Application Programming Interface (REST API) merupakan pendekatan arsitektur yang digunakan untuk menghubungkan aplikasi client dengan server melalui protokol HTTP. REST API memanfaatkan konsep resource yang diakses menggunakan Uniform Resource Identifier (URI), sedangkan proses komunikasi dilakukan melalui metode HTTP seperti GET, POST, PUT, dan DELETE.

Pendekatan REST memiliki karakteristik stateless, yaitu setiap request yang dikirim client harus membawa seluruh informasi yang diperlukan sehingga server tidak menyimpan status komunikasi sebelumnya. Karakteristik ini memberikan keuntungan berupa kemudahan dalam pengembangan, peningkatan skalabilitas sistem, serta kemudahan integrasi dengan berbagai platform, termasuk aplikasi mobile.

Pada penelitian ini, REST API digunakan sebagai media komunikasi antara aplikasi Flutter dan backend Laravel. Seluruh proses pengelolaan data, seperti autentikasi pengguna, pengelolaan data pasien, dokter, jadwal pemeriksaan, antrian, serta rekam medis dilakukan melalui endpoint REST API yang mengirim dan menerima data dalam format JavaScript Object Notation (JSON). Pendekatan tersebut memungkinkan aplikasi mobile dan backend dikembangkan secara terpisah namun tetap terintegrasi melalui layanan API yang sama.

C. HTTP Protocol

Hypertext Transfer Protocol (HTTP) merupakan protokol komunikasi yang digunakan untuk melakukan pertukaran data antara client dan server pada aplikasi berbasis web maupun mobile. Dalam mekanisme komunikasi ini, client mengirimkan HTTP Request kepada server, kemudian server memproses permintaan tersebut dan mengembalikan HTTP Response yang berisi informasi mengenai hasil pemrosesan.

Setiap HTTP Request terdiri atas beberapa komponen, seperti metode HTTP, URI, header, dan message body, sedangkan HTTP Response berisi status code, header, dan body yang umumnya dikirimkan dalam format JSON pada implementasi REST API. Penggunaan status code, seperti 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, dan

404 Not Found, membantu client dalam mengetahui hasil pemrosesan setiap request yang dikirimkan.

Pada penelitian ini, komunikasi antara aplikasi Flutter dan Laravel dilakukan menggunakan protokol HTTP melalui REST API. Seluruh data yang dikirim maupun diterima menggunakan format JSON sehingga proses pertukaran informasi menjadi ringan, cepat, dan mudah diproses oleh kedua platform.

D. MVC Architecture

Model–View–Controller (MVC) merupakan salah satu pola arsitektur perangkat lunak yang bertujuan untuk memisahkan tanggung jawab setiap komponen aplikasi sehingga proses pengembangan, pemeliharaan, dan pengujian sistem menjadi lebih mudah. Arsitektur ini membagi aplikasi menjadi tiga komponen utama, yaitu Model, View, dan Controller.

Model bertanggung jawab terhadap pengelolaan data dan logika bisnis aplikasi. Komponen ini berinteraksi langsung dengan basis data untuk melakukan proses penyimpanan, pengambilan, pembaruan, maupun penghapusan data. View berfungsi sebagai antarmuka pengguna yang menampilkan informasi kepada pengguna berdasarkan data yang diperoleh dari Model. Sementara itu, Controller bertugas menerima permintaan dari pengguna, memproses logika aplikasi, berkomunikasi dengan Model, kemudian mengembalikan hasilnya kepada View.

Penerapan pola MVC memberikan beberapa keuntungan, antara lain meningkatkan modularitas aplikasi, mempermudah proses pemeliharaan, mengurangi keterkaitan antar komponen (low coupling), serta memungkinkan pengembangan dilakukan secara lebih terstruktur. Oleh karena itu, pola MVC banyak diterapkan pada framework pengembangan web modern, termasuk Laravel, untuk menghasilkan aplikasi yang lebih mudah dikembangkan dan memiliki struktur kode yang lebih baik.

E. Laravel Framework

Laravel merupakan framework berbasis PHP yang menerapkan arsitektur Model–View–Controller (MVC) untuk membantu pengembangan aplikasi web secara lebih terstruktur. Framework ini menyediakan berbagai fitur yang mendukung pengembangan aplikasi modern, seperti sistem routing, middleware, autentikasi, migrasi basis data, Eloquent ORM, serta Resource Controller untuk pembangunan REST API.

Melalui sistem routing, Laravel mampu mengarahkan setiap permintaan HTTP menuju controller yang sesuai. Selanjutnya, controller memproses logika bisnis aplikasi dan berinteraksi dengan model melalui Eloquent ORM untuk melakukan operasi terhadap basis data. Hasil pemrosesan kemudian dikembalikan kepada client dalam bentuk respons, yang pada implementasi REST API umumnya menggunakan format JSON.

Pada penelitian ini, Laravel digunakan sebagai backend yang mengelola seluruh proses bisnis aplikasi KlinikKu. Framework ini menangani autentikasi pengguna menggunakan Laravel Sanctum, validasi data, pengelolaan endpoint REST API, serta proses Create, Read, Update, dan Delete (CRUD) terhadap data pasien, dokter, jadwal pemeriksaan, antrean, dan rekam medis yang tersimpan pada basis data MySQL.

F. Flutter Framework

Flutter merupakan framework pengembangan aplikasi mobile berbasis open-source yang dikembangkan oleh Google. Framework ini menggunakan bahasa pemrograman Dart dan menerapkan konsep cross-platform development, sehingga satu basis kode dapat digunakan untuk membangun aplikasi pada berbagai sistem operasi, seperti Android dan iOS.

Flutter menyediakan berbagai komponen antarmuka (widget) yang memungkinkan pengembang membangun aplikasi dengan tampilan yang konsisten serta performa yang tinggi. Selain itu, fitur hot reload mempercepat proses pengembangan karena perubahan kode dapat langsung ditampilkan tanpa harus membangun ulang aplikasi secara keseluruhan.

Pada penelitian ini, Flutter berperan sebagai aplikasi client yang digunakan oleh admin atau petugas klinik untuk mengakses layanan sistem. Seluruh interaksi pengguna, seperti login, pengelolaan data pasien, dokter, jadwal pemeriksaan, antrean, dan rekam medis dilakukan melalui komunikasi dengan backend Laravel menggunakan REST API. Data yang diterima dari server dalam format JSON kemudian diproses dan ditampilkan melalui antarmuka aplikasi sehingga pengguna memperoleh informasi secara cepat dan terintegrasi.

G. Related Works

Beberapa penelitian sebelumnya telah menerapkan arsitektur client–server dan REST API dalam pengembangan sistem informasi berbasis web maupun mobile. Penelitian mengenai implementasi RESTful API menggunakan Laravel dan Flutter menunjukkan bahwa pemisahan antara aplikasi client dan server mampu meningkatkan fleksibilitas pengembangan serta mempermudah integrasi antar platform. Pendekatan tersebut memungkinkan aplikasi mobile berkomunikasi dengan backend secara efisien melalui pertukaran data dalam format JSON.

Penelitian lain mengenai sistem informasi rekam medis klinik berbasis web menunjukkan bahwa digitalisasi proses administrasi mampu meningkatkan efektivitas pengelolaan data pasien, dokter, jadwal pemeriksaan, serta riwayat rekam medis. Sistem tersebut berhasil menggantikan proses pencatatan manual sehingga informasi menjadi lebih terstruktur dan mudah diakses.

Berdasarkan penelitian-penelitian tersebut, pengembangan KlinikKu mengombinasikan penerapan arsitektur client–server, REST API, framework Flutter, dan Laravel dalam sebuah sistem informasi klinik berbasis mobile. Berbeda dengan penelitian sebelumnya yang umumnya berfokus pada aplikasi berbasis web atau hanya membahas komunikasi REST API, penelitian ini mengintegrasikan pengelolaan pasien, dokter, jadwal pemeriksaan, antrean, dan rekam medis dalam satu aplikasi mobile yang terhubung dengan backend Laravel melalui REST API. Dengan demikian, penelitian ini diharapkan dapat memberikan alternatif implementasi sistem informasi klinik yang lebih fleksibel, terintegrasi, dan mudah dikembangkan.

III. RESEARCH METHOD

A. Research Flow

Penelitian ini menggunakan tahapan pengembangan sistem yang dimulai dari identifikasi permasalahan hingga evaluasi hasil implementasi. Tahap pertama adalah identifikasi masalah, yaitu menganalisis kendala yang terjadi pada proses administrasi klinik, seperti pengelolaan data pasien, jadwal pemeriksaan, antrian, dan rekam medis yang masih dilakukan secara manual. Selanjutnya dilakukan analisis kebutuhan sistem untuk menentukan kebutuhan fungsional maupun nonfungsional yang akan diimplementasikan pada aplikasi.

Tahap berikutnya adalah perancangan sistem, yang meliputi perancangan arsitektur client-server, struktur basis data, desain REST API, serta penerapan pola Model-View-Controller (MVC) pada backend Laravel dan arsitektur aplikasi Flutter pada sisi client. Setelah proses perancangan selesai, dilakukan implementasi sistem menggunakan Flutter sebagai aplikasi mobile, Laravel sebagai backend REST API, dan MySQL sebagai basis data.

Tahap selanjutnya adalah pengujian sistem menggunakan metode Black Box Testing untuk memastikan setiap fitur berjalan sesuai dengan kebutuhan fungsional. Hasil pengujian kemudian dianalisis pada tahap evaluasi untuk mengetahui apakah sistem telah memenuhi tujuan penelitian.

B. Requirement Analysis

1) Functional Requirements

Berdasarkan hasil analisis kebutuhan, sistem KlinikKu memiliki beberapa kebutuhan fungsional sebagai berikut.

- Pengguna dapat melakukan proses autentikasi melalui fitur login dan logout.
- Sistem dapat menampilkan informasi ringkasan pada halaman dashboard.
- Sistem dapat melakukan pengelolaan data pasien meliputi penambahan, perubahan, penghapusan, dan penampilan data.
- Sistem dapat melakukan pengelolaan data dokter.
- Sistem dapat melakukan pengelolaan jadwal pemeriksaan.
- Sistem dapat melakukan pengelolaan data antrian pasien.
- Sistem dapat melakukan pengelolaan data rekam medis.
- Sistem melakukan pertukaran data antara aplikasi Flutter dan Laravel menggunakan REST API.

2) Non-Functional Requirements

Kebutuhan nonfungsional sistem meliputi penggunaan Flutter sebagai framework pengembangan aplikasi mobile, Laravel 13 sebagai framework backend, MySQL sebagai sistem manajemen basis data, serta REST API berbasis HTTP dengan format pertukaran data JSON. Selain itu, sistem menerapkan Laravel Sanctum sebagai mekanisme autentikasi API dan pola arsitektur Model-View-Controller (MVC) pada backend untuk meningkatkan keteraturan pengembangan perangkat lunak.

C. System Architecture

Arsitektur sistem yang digunakan pada penelitian ini menerapkan konsep client-server, di mana aplikasi Flutter berperan sebagai client dan Laravel sebagai server yang

menyediakan layanan REST API. Seluruh komunikasi data dilakukan menggunakan protokol HTTP dengan format pertukaran data JSON.

Pada sisi client, pengguna berinteraksi melalui halaman Flutter (Pages). Setiap halaman menggunakan Service untuk melakukan komunikasi HTTP dengan REST API Laravel. Data yang diterima dari server kemudian dipetakan menggunakan Model Dart sebelum ditampilkan pada antarmuka aplikasi. Selanjutnya, route Laravel mengarahkan request ke controller yang sesuai untuk menjalankan logika bisnis aplikasi. Controller kemudian berinteraksi dengan model yang mengelola akses ke basis data MySQL. Setelah proses selesai, Controller Laravel mengembalikan hasil pemrosesan dalam bentuk response JSON yang dikirim kembali ke aplikasi Flutter melalui service. Data JSON tersebut kemudian diproses oleh aplikasi Flutter sebelum ditampilkan kepada pengguna. Data tersebut diproses oleh controller Flutter sebelum ditampilkan pada halaman aplikasi.

Dengan pendekatan ini, proses pengembangan frontend dan backend dapat dilakukan secara terpisah tanpa mengurangi integrasi antar komponen sistem. Arsitektur tersebut juga mempermudah proses pemeliharaan, pengembangan fitur baru, serta meningkatkan skalabilitas aplikasi.

D. MVC Architecture

Arsitektur aplikasi pada penelitian ini menerapkan pola Model-View-Controller (MVC) pada backend Laravel yang dipadukan dengan arsitektur aplikasi Flutter sebagai client. Pendekatan ini bertujuan untuk memisahkan tanggung jawab setiap komponen sehingga proses pengembangan, pemeliharaan, dan pengujian aplikasi menjadi lebih mudah dilakukan.

Pada sisi backend Laravel, komponen Model bertanggung jawab terhadap pengelolaan data dan interaksi dengan basis data MySQL melalui Eloquent ORM. Model yang digunakan meliputi User, Pasien, Dokter, Jadwal, Antrian, dan RekamMedis. Seluruh proses Create, Read, Update, dan Delete (CRUD) dilakukan melalui model tersebut.

Komponen Controller bertugas menerima permintaan dari client, menjalankan logika bisnis aplikasi, melakukan validasi data, serta berinteraksi dengan model. Controller yang digunakan terdiri atas AuthController, DashboardController, PasienController, DokterController, JadwalController, AntrianController, dan RekamMedisController.

Setelah data diproses oleh Controller, Laravel mengirimkan hasil pemrosesan kepada client dalam bentuk response JSON. Response tersebut berisi data yang diperlukan oleh aplikasi Flutter untuk ditampilkan pada halaman pengguna. Penggunaan format JSON memungkinkan proses pertukaran data antara backend Laravel dan aplikasi Flutter berjalan secara sederhana dan terstruktur. Seluruh endpoint REST API dihubungkan dengan controller melalui berkas routes/api.php.

Pada sisi Flutter, aplikasi menerapkan struktur yang terdiri atas Model, Service, Controller, dan View (Pages). Model digunakan untuk merepresentasikan objek yang diterima dari respons JSON. Service bertanggung jawab melakukan komunikasi HTTP dengan REST API Laravel. Controller mengelola logika aplikasi dan perubahan state, sedangkan View menampilkan antarmuka pengguna serta menerima masukan dari pengguna. Struktur ini memungkinkan

pemisahan fungsi setiap komponen sehingga aplikasi menjadi lebih modular dan mudah dikembangkan.

E. Database Design

Basis data yang digunakan pada penelitian ini adalah MySQL. Perancangan basis data dilakukan untuk mendukung seluruh proses bisnis pada sistem informasi klinik, mulai dari autentikasi pengguna hingga pengelolaan rekam medis.

Struktur basis data terdiri atas beberapa entitas utama, yaitu User, Pasien, Dokter, Jadwal, Antrian, dan RekamMedis. Masing-masing entitas direpresentasikan dalam bentuk model Laravel sehingga proses interaksi dengan basis data dilakukan menggunakan Eloquent ORM.

Hubungan antarentitas dirancang agar mendukung proses pelayanan klinik secara terintegrasi. Data pasien dapat memiliki beberapa riwayat rekam medis, dokter memiliki jadwal praktik yang tersimpan dalam tabel jadwal, sedangkan antrian pasien menghubungkan data pasien dengan jadwal pelayanan yang tersedia. Seluruh data tersebut saling berelasi sehingga proses pengelolaan informasi dapat dilakukan secara konsisten melalui REST API.

Perancangan basis data ini bertujuan untuk meminimalkan redundansi data, menjaga integritas informasi, serta mempermudah proses pengambilan maupun pembaruan data pada setiap modul aplikasi.

F. REST API Design

Komunikasi antara aplikasi Flutter dan backend Laravel dilakukan menggunakan REST API dengan protokol HTTP serta format pertukaran data JSON. Setiap fitur pada aplikasi diimplementasikan melalui endpoint yang dihubungkan dengan controller Laravel menggunakan berkas routes/api.php.

Layanan autentikasi terdiri atas endpoint POST /api/login dan POST /api/logout untuk mengelola proses masuk dan keluar pengguna. Setelah berhasil melakukan autentikasi, pengguna dapat mengakses berbagai layanan yang disediakan oleh sistem.

Modul Dashboard menyediakan endpoint GET /api/dashboard untuk menampilkan ringkasan informasi sistem. Selanjutnya, setiap modul utama menerapkan operasi CRUD melalui endpoint REST API. Modul Pasien, Dokter, Jadwal, Antrian, dan Rekam Medis masing-masing menyediakan endpoint GET untuk menampilkan data, POST untuk menambahkan data baru, PUT untuk memperbarui data yang sudah ada, serta DELETE untuk menghapus data.

Perancangan endpoint yang konsisten memudahkan aplikasi Flutter dalam mengakses seluruh layanan backend. Selain itu, penggunaan metode HTTP sesuai standar REST API membuat proses integrasi antarplatform menjadi lebih sederhana, mudah dipelihara, dan mudah dikembangkan apabila di masa mendatang diperlukan penambahan fitur baru.

G. Technology Stack

Pengembangan aplikasi KlinikKu memanfaatkan beberapa teknologi yang saling terintegrasi. Framework Flutter digunakan sebagai pengembang aplikasi mobile berbasis Android, sedangkan Laravel 13 digunakan sebagai backend yang menyediakan layanan REST API. Komunikasi antara frontend dan backend dilakukan menggunakan protokol HTTP dengan format pertukaran data JSON.

Sistem menggunakan MySQL sebagai basis data untuk menyimpan seluruh informasi mengenai pengguna, pasien, dokter, jadwal, antrian, dan rekam medis. Pada sisi backend, Laravel Sanctum digunakan sebagai mekanisme autentikasi API, sedangkan Eloquent ORM dimanfaatkan untuk mempermudah proses interaksi antara aplikasi dan basis data.

Selama proses pengembangan, pengujian endpoint REST API dilakukan menggunakan Thunder Client pada Visual Studio Code untuk memastikan setiap endpoint dapat berjalan sesuai kebutuhan, sedangkan Laragon digunakan sebagai lingkungan pengembangan lokal untuk menjalankan layanan Apache, PHP, dan MySQL. Seluruh kode sumber aplikasi dikelola menggunakan Git sehingga proses pengembangan dapat dilakukan secara kolaboratif dan terdokumentasi dengan baik.

IV. RESULTS AND DISCUSSION

A. Hasil Implementasi Sistem

Aplikasi KlinikKu berhasil dikembangkan sebagai sistem informasi klinik berbasis mobile dengan menerapkan arsitektur client-server. Sistem terdiri dari dua komponen utama, yaitu aplikasi Flutter sebagai client dan Laravel 13 sebagai backend API server. Flutter bertanggung jawab dalam menyediakan antarmuka pengguna serta melakukan pertukaran data melalui REST API, sedangkan Laravel menangani proses bisnis aplikasi, autentikasi pengguna, pengelolaan data, serta komunikasi dengan database MySQL.

Pengembangan sistem menerapkan pemisahan tanggung jawab berdasarkan arsitektur Model-View-Controller (MVC) pada backend Laravel. Komponen Model digunakan untuk merepresentasikan data dan mengelola interaksi dengan database menggunakan Eloquent ORM, Controller menangani proses request dari client serta menjalankan logika aplikasi, sedangkan Controller Laravel bertanggung jawab menghasilkan response dalam format JSON yang digunakan sebagai hasil komunikasi antara backend dan aplikasi Flutter.

Pada sisi Flutter, aplikasi menggunakan struktur modular yang terdiri dari konfigurasi API, Model, Pages, dan Service. Pages digunakan sebagai antarmuka pengguna, Service bertanggung jawab melakukan komunikasi HTTP dengan REST API Laravel, sedangkan Model digunakan untuk melakukan pemetaan data JSON menjadi objek Dart. Struktur tersebut membuat proses pengembangan aplikasi menjadi lebih terorganisasi dan mudah dilakukan pemeliharaan.

B. Implementasi Backend Laravel 13

Backend aplikasi KlinikKu dikembangkan menggunakan framework Laravel 13 yang berfungsi sebagai penyedia layanan REST API. Backend menangani proses autentikasi pengguna, pengelolaan data klinik, serta menyediakan endpoint yang dapat digunakan oleh aplikasi Flutter.

Struktur backend menggunakan konsep MVC dengan beberapa komponen utama yaitu:

C. Model

Model digunakan untuk menghubungkan aplikasi dengan tabel database. Model yang digunakan dalam sistem KlinikKu terdiri dari:

TABEL I. MODEL

Model	Fungsi
User	Mengelola data pengguna dan hak akses
Pasien	Mengelola informasi pasien
Dokter	Mengelola informasi dokter
JadwalPemeriksaan	Mengelola jadwal pemeriksaan
Antrian	Mengelola data antrean pasien
RekamMedis	Mengelola riwayat pemeriksaan pasien

Model bertanggung jawab dalam mengatur hubungan antar tabel menggunakan fitur Eloquent ORM Laravel.

D. Controller

Controller digunakan sebagai penghubung antara request dari aplikasi Flutter dengan database.

Controller yang digunakan:

TABEL III. DATABASE

Controller	Fungsi
AuthController	Proses login, register, logout
DashboardController	Menampilkan data statistik dashboard
PasienController	CRUD data pasien
DokterController	CRUD data dokter
JadwalController	CRUD jadwal pemeriksaan
AntrianController	Mengatur proses antrean
RekamMedisController	Mengelola data rekam medis

Controller menerima request HTTP dari client kemudian menjalankan logika aplikasi sebelum memberikan response JSON.

V. IMPLEMENTASI AUTHENTICATION

Sistem autentikasi pada KlinikKu menggunakan Laravel Sanctum sebagai mekanisme keamanan API.

Proses autentikasi dilakukan melalui beberapa tahapan:

1. Pengguna memasukkan email dan password melalui halaman login Flutter.
2. Data dikirim menggunakan metode HTTP POST menuju endpoint: POST /api/login
3. Laravel melakukan validasi terhadap data pengguna pada tabel users.
4. Jika data valid, sistem membuat token autentikasi menggunakan Laravel Sanctum.
5. Token dikirim kembali ke aplikasi Flutter.
6. Token disimpan menggunakan SharedPreferences.

Alur autentikasi dapat dilihat pada Gambar 2.

Gambar 2. Alur Login KlinikKu

Penerapan Laravel Sanctum memberikan keamanan tambahan karena setiap request yang membutuhkan autentikasi harus menyertakan token akses yang valid.

VI. IMPLEMENTASI REST API

REST API digunakan sebagai media komunikasi antara Flutter sebagai client dengan Laravel sebagai server.

Endpoint utama yang digunakan pada sistem KlinikKu ditunjukkan pada Tabel I.

TABEL II. Endpoint REST API KlinikKu

Modul	Endpoint	Method
Register	/api/register	POST
Login	/api/login	POST
Logout	/api/logout	POST
Dashboard	/api/dashboard	GET
Pasien	/api/pasien	GET, POST
Pasien	/api/pasien/{id}	PUT, DELETE
Dokter	/api/dokter	GET, POST
Dokter	/api/dokter/{id}	PUT, DELETE
Jadwal	/api/jadwal-pemeriksaan	GET, POST
Jadwal	/api/jadwal-pemeriksaan/{id}	PUT, DELETE
Antrian	/api/antrian	GET, POST
Antrian	/api/antrian/{id}	PUT, DELETE
Rekam Medis	/api/rekam-medis	GET, POST
Rekam Medis	/api/rekam-medis/{id}	PUT, DELETE

A. Implementasi Database

Database KlinikKu menggunakan MySQL dengan beberapa tabel utama:

TABEL IV. FUNGSI DATA BASE

Tabel	Fungsi
users	Menyimpan data pengguna
pasien	Menyimpan data pasien
dokter	Menyimpan data dokter
jadwal_pemeriksaan	Menyimpan jadwal pemeriksaan
antrian	Menyimpan data antrean
rekam_medis	Menyimpan riwayat medis pasien

Implementasi Flutter Client

Aplikasi Flutter KlinikKu menggunakan struktur modular yang terdiri dari:

- 1) Config

Folder config digunakan untuk menyimpan konfigurasi API.

Contoh:

api.dart

api_config.dart

Komponen ini digunakan untuk menentukan alamat server API yang digunakan aplikasi.

2) Models

Model Flutter digunakan untuk mengubah data JSON dari REST API menjadi objek Dart.

Model yang digunakan:

Model	Fungsi
user.dart	Data pengguna
pasien.dart	Data pasien
dokter.dart	Data dokter
jadwal_pemeriksaan.dart	Data jadwal
antrian.dart	Data antrean
rekam_medis.dart	Data rekam medis

3) Pages

Pages berfungsi sebagai View pada aplikasi Flutter.

Halaman yang tersedia:

Halaman	Fungsi
Login Page	Autentikasi pengguna
Register Page	Registrasi pengguna
Home Page	Dashboard aplikasi
Pasien Page	Pengelolaan pasien
Dokter Page	Pengelolaan dokter
Jadwal Page	Pengelolaan jadwal
Antrian Page	Pengelolaan antrean
Rekam Medis Page	Pengelolaan rekam medis

4) Services

Service digunakan untuk menangani komunikasi HTTP antara Flutter dan Laravel API.

Contoh service:

auth_service.dart

pasien_service.dart

dokter_service.dart

jadwal_service.dart

antrian_service.dart

rekam_medis_service.dart

Setiap service bertanggung jawab mengirim request dan menerima response dari endpoint tertentu.

VII. IMPLEMENTASI FITUR SISTEM

A. Dashboard

Dashboard menampilkan informasi utama klinik berupa:

- Total pasien
- Total dokter
- Jumlah antrean menunggu
- Jumlah antrean dipanggil
- Pemeriksaan selesai hari ini
- Jadwal pemeriksaan hari ini

Dashboard membantu pengguna memperoleh informasi kondisi klinik secara cepat.

B. Pengelolaan Pasien

Fitur pasien menyediakan fungsi:

- Menampilkan data pasien
- Menambahkan pasien baru
- Mengubah data pasien
- Menghapus data pasien

Data pasien yang dikelola:

- Nama
- Tanggal lahir
- Jenis kelamin
- Alamat
- Nomor telepon

C. Pengelolaan Dokter

Modul dokter digunakan untuk mengelola informasi tenaga medis.

Data dokter:

- Nama dokter
- Spesialisasi
- Nomor telepon

Fitur:

- Tambah dokter
- Edit dokter
- Hapus dokter
- Melihat daftar dokter

D. Pengelolaan Jadwal Pemeriksaan

Modul jadwal digunakan untuk mengatur pemeriksaan pasien.

Data yang disimpan:

- Pasien
- Dokter
- Tanggal pemeriksaan
- Waktu pemeriksaan
- Keluhan pasien

E. Pengelolaan Antrian

Sistem antrean menggunakan nomor tiket seperti:

A001

A002

A003

Status antrean:

- Menunggu
- Dipanggil
- Selesai

Fitur ini membantu staff klinik mengatur proses pelayanan pasien.

F. Rekam Medis

Modul rekam medis menyimpan riwayat pemeriksaan pasien.

Data yang dikelola:

- Pasien
- Dokter
- Tanggal pemeriksaan
- Diagnosis
- Resep
- Catatan

VIII. PENGUJIAN SISTEM

Pengujian dilakukan menggunakan Thunder Client untuk memastikan seluruh endpoint REST API berjalan sesuai kebutuhan.

Metode pengujian yang digunakan adalah Black Box Testing dengan fokus pada input dan output sistem.

Tabel V. Hasil Pengujian API

Pengujian	Endpoint	Hasil
Register pengguna	POST /api/register	Berhasil
Login pengguna	POST /api/login	Berhasil
Mengambil data pasien	GET /api/pasien	Berhasil
Menambah pasien	POST /api/pasien	Berhasil
Mengubah pasien	PUT /api/pasien/{id}	Berhasil

Menghapus pasien	DELETE /api/pasien/{id}	Berhasil
Mengambil data dokter	GET /api/dokter	Berhasil
Mengelola antrean	API Antrian	Berhasil
Mengelola rekam medis	API Rekam Medis	Berhasil

REFERENCES

- [1] U. Manu, R. Noviana, and J. M. R. No, "Analisis kualitas aplikasi Unit Link menggunakan metode ISO 25010 (studi kasus PT Asuransi Jiwasraya Persero)," *Jurnal Ilmiah MATRIK*, vol. 24, no. 2, 2022.
- [2] Z. Subecz, "Web-development with Laravel framework," *Gradus*, vol. 8, no. 1, pp. 211–218, 2021.
- [3] J. Jasri, "Arsitektur three tier (client server programing) pada aplikasi perpustakaan Fakultas Teknik Universitas Muhammadiyah Bengkulu," *Journal of Technopreneurship and Information System*, vol. 1, no. 1, pp. 19–25, 2018.
- [4] A. Suryadi, Y. W. T. Arif, and N. S. Novitasari, "Rancang bangun sistem informasi rekam medis klinik rawat jalan berbasis web," *Infokes: Jurnal Ilmiah Rekam Medis dan Informatika Kesehatan*, vol. 12, no. 1, pp. 37–43, 2022.
- [5] D. P. Pop and A. Altar, "Designing an MVC model for rapid web application development," *Procedia Engineering*, vol. 69, pp. 1172–1179, 2014.
- [6] V. Surwase, "REST API modeling languages—A developer's perspective," *International Journal of Science Technology & Engineering*, vol. 2, no. 10, pp. 634–637, 2016.
- [7] A. Sunardi and Suhajito, "MVC architecture: A comparative study between Laravel framework and Slim framework in freelancer project monitoring system web based," *Procedia Computer Science*, vol. 157, pp. 134–141, 2019.
- [8] R. Sridaran, G. Padmavathi, K. Iyakutti, and M. N. S. Mani, "SPIM architecture for MVC based web applications," *Journal of Advanced Networking and Applications*, vol. 1, no. 1, pp. 63–68, 2010.
- [9] J. Deacon, *Model-view-controller (MVC) architecture*. 2009. [Online]. Available: <http://www.jdl.co.uk/briefings/MVC.pdf>
- [11] S. Kosasi, "Perancangan aplikasi point of sale dengan arsitektur client/server berbasis Linux dan Windows," *Creative Information Technology Journal*, vol. 1, no. 2, pp. 116–127, 2014.
- [12] R. Fielding, M. Nottingham, and J. Reschke, Eds., *RFC 9110: HTTP Semantics*, RFC 9110, Internet Engineering Task Force (IETF), Jun. 2022.